Universidad del Valle de Guatemala
Facultad de Ciencias y Humanidades
Lógica Matemática

Tarea Prolog:

Árbol genealógico y caminos en un grafo

Trabajo realizado por:
17444 – Juan Menéndez
19644 – Luis Mendoza
231169 – Mishell Ciprian
241040 – Diana Sosa
24290 – René Abella

Docente:
Paulo Mejía

Guatemala, 25 de octubre de 2026

1. Desarrollo

1.1. Ejercicio 1: hechos del árbol genealógico

Enunciado. Construir el árbol genealógico mediante los hechos `hombre/1`, `mujer/1` y `progenitor/2`, y representarlo gráficamente. Para el trabajo se utilizó la familia Simpson.

Descripción del árbol. El árbol tiene tres generaciones. En la primera están Abraham y Mona, padres de Homero, y Clancy y Jacqueline, padres de Marge, Patty y Selma. En la segunda están Homero y Marge, que son pareja, junto con Patty, Selma y Herb, que es hijo de Abraham y medio hermano de Homero. En la tercera están Bart, Lisa y Maggie, hijos de Homero y Marge, y Ling, hija adoptiva de Selma. Se incluyó a Herb y a Ling para que todas las relaciones pedidas tuvieran al menos un caso en el árbol.

Código en Prolog: Los hechos se escribieron al inicio del archivo `familia.pl`.

```
% hombre(X): X es hombre.
hombre(abraham).
hombre(clancy).
hombre(herb).
hombre(homero).
hombre(bart).

% mujer(X): X es mujer.
mujer(mona).
mujer(jacqueline).
mujer(marge).
mujer(patty).
mujer(selma).
mujer(lisa).
mujer(maggie).
mujer(ling).

% progenitor(X, Y): X es padre o madre de Y.
progenitor(abraham, homero).
progenitor(abraham, herb).
progenitor(mona, homero).

progenitor(clancy, marge).
progenitor(clancy, patty).
progenitor(clancy, selma).
progenitor(jacqueline, marge).
progenitor(jacqueline, patty).
progenitor(jacqueline, selma).

progenitor(homero, bart).
progenitor(homero, lisa).
progenitor(homero, maggie).
```

```
progenitor(marge, bart).
progenitor(marge, lisa).
progenitor(marge, maggie).

progenitor(selma, ling).
```

Explicación paso a paso.

- `hombre(X)` y `mujer(X)` indican si una persona es hombre o mujer. Cada persona aparece en uno solo de los dos grupos.
- `progenitor(X, Y)` indica que X es padre o madre de Y. Por eso cada hijo de Homero y Marge tiene dos hechos, uno con cada progenitor.
- Herb y Ling tienen un solo progenitor registrado, porque el otro no forma parte del árbol.
- Los nombres se escriben en minúscula y sin tildes, porque en Prolog una palabra que empieza con mayúscula se interpreta como una variable.
- Solo se registran estos tres tipos de hechos. Todas las demás relaciones del ejercicio se obtienen a partir de ellos mediante reglas.

Representación gráfica.

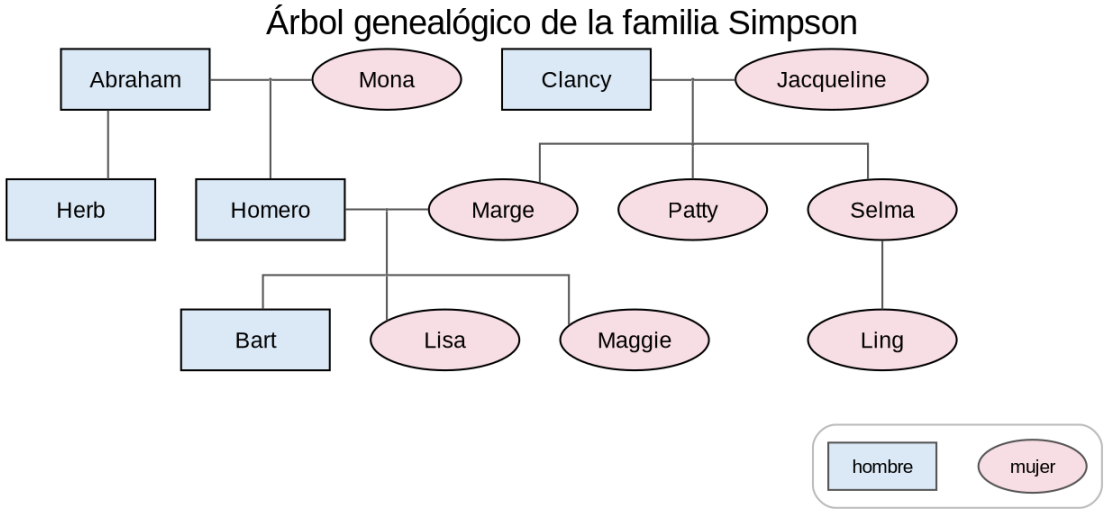


Figura 1: Árbol genealógico de la familia Simpson. Los rectángulos representan hombres y las elipses, mujeres.

En la figura 1, cada línea horizontal une a una pareja y de ella bajan las líneas hacia sus hijos. Herb está unido solo a Abraham y Ling solo a Selma, porque son los únicos progenitores registrados para ellos.

Consultas y resultados. Se realizaron las siguientes consultas en SWI-Prolog:

```
?- progenitor(X, bart).
X = homero ;
X = marge.

?- progenitor(abraham, X).
X = homero ;
X = herb.

?- mujer(ling).
true.

?- hombre(maggie).
false.
```

La primera consulta devuelve los dos progenitores de Bart y la segunda, los dos hijos de Abraham. La última respondió `false` porque Maggie está registrada como mujer.

1.2. Ejercicio 1: relaciones `padre`, `madre` e `hijo`

Enunciado. Definir las relaciones `padre(X, Y)`, `madre(X, Y)` e `hijo(X, Y)`, utilizando los hechos `progenitor/2`, `hombre/1` y `mujer/1` del archivo `familia.pl`.

Razonamiento en lenguaje natural. Una persona es padre de otra si es su progenitor y es hombre. Una persona es madre de otra si es su progenitor y es mujer. Para determinar si X es hijo de Y, se comprueba que Y sea progenitor de X y que X sea hombre.

Código en Prolog:

```
% padre(X, Y): X es padre de Y.
padre(X, Y) :-
    progenitor(X, Y),
    hombre(X).

% madre(X, Y): X es madre de Y.
madre(X, Y) :-
    progenitor(X, Y),
    mujer(X).

% hijo(X, Y): X es hijo de Y.
hijo(X, Y) :-
    progenitor(Y, X),
    hombre(X).
```

Explicación paso a paso.

- En `padre`, se comprueba que X sea progenitor de Y y que X sea hombre.

- En `madre`, se comprueba que X sea progenitor de Y y que X sea mujer.
- En `hijo`, se utiliza `progenitor(Y, X)` porque X es el hijo y Y es su progenitor. También se comprueba que X sea hombre.
- La coma significa que ambas condiciones deben cumplirse para que la relación sea verdadera.

Consultas y resultados. Se realizaron las siguientes consultas en SWI-Prolog:

```
?- padre(homero, bart).
true.

?- madre(marge, lisa).
true.

?- hijo(bart, homero).
true.

?- hijo(lisa, homero).
false.
```

Las primeras tres consultas respondieron `true`, de acuerdo con los hechos registrados en el árbol. La última respondió `false` porque Lisa es mujer y la regla `hijo` requiere que el primer argumento sea un hombre.

1.3. Ejercicio 1: relaciones `hija`, `abuelo` y `abuela`

Enunciado. Definir las relaciones `hija(X, Y)`, `abuelo(X, Y)` y `abuela(X, Y)`, utilizando los hechos `progenitor/2`, `hombre/1` y `mujer/1` del archivo `familia.pl`.

Razonamiento en lenguaje natural. X es hija de Y si Y es progenitor de X y X es mujer. X es abuelo de Y si X es progenitor de alguno de los progenitores de Y y X es hombre. La relación abuela se define de la misma forma, pero X debe ser mujer. Por ejemplo, Clancy es progenitor de Selma y Selma es progenitora de Ling, por lo que Clancy es abuelo de Ling.

Código en Prolog: Las reglas se agregaron al archivo `familia.pl`, después de las reglas anteriores.

```
% hija(X, Y): X es hija de Y (hija primero, progenitor despues).
hija(X, Y) :-
    progenitor(Y, X),
    mujer(X).

% abuelo(X, Y): X es abuelo de Y por cualquiera de sus progenitores.
abuelo(X, Y) :-
    progenitor(X, Z),
    progenitor(Z, Y),
```

```
    hombre(X).

% abuela(X, Y): X es abuela de Y por cualquiera de sus progenitores.
abuela(X, Y) :-
    progenitor(X, Z),
    progenitor(Z, Y),
    mujer(X).
```

Explicación paso a paso.

- En `hija`, se utiliza `progenitor(Y, X)` porque Y es el progenitor y X es la hija. Después, `mujer(X)` comprueba que X sea mujer.
- En `abuelo`, `progenitor(X, Z)` y `progenitor(Z, Y)` buscan una persona Z que sea hija o hijo de X y, a la vez, progenitor de Y. Z representa la generación intermedia.
- Como Z puede ser el padre o la madre de Y, la regla encuentra a los abuelos de ambos lados de la familia.
- `hombre(X)` en `abuelo` y `mujer(X)` en `abuela` distinguen entre abuelo y abuela.

Consultas y resultados. Se realizaron las siguientes consultas en SWI-Prolog:

```
?- hija(lisa, homero).
true.

?- hija(bart, homero).
false.

?- abuelo(abraham, bart).
true ;
false.

?- abuela(jacqueline, ling).
true.

?- abuelo(X, ling).
X = clancy ;
false.

?- abuela(mona, ling).
false.
```

Cuando Prolog responde `true ;` seguido de `false`, significa que encontró la respuesta y que, al pedirle otra con `;`, ya no encontró otra forma de demostrarla. La consulta `hija(bart, homero)` respondió `false` porque Bart es hombre, y `abuela(mona, ling)` respondió `false` porque Mona es abuela por el lado de Homero, mientras que Ling es hija de Selma.

Además, con `setof/3` se obtuvieron todas las parejas de cada relación:

Consulta	Resultado	Explicación
`hija(X, Y)`	11 parejas	Lisa y Maggie con Homero y Marge (4); Marge, Patty y Selma con Clancy y Jacqueline (6); Ling con Selma (1).
`abuelo(X, Y)`	7 parejas	Abraham es abuelo de Bart, Lisa y Maggie; Clancy es abuelo de Bart, Lisa, Maggie y Ling.
`abuela(X, Y)`	7 parejas	Mona es abuela de Bart, Lisa y Maggie; Jacqueline es abuela de Bart, Lisa, Maggie y Ling.

1.4. Ejercicio 1: relaciones `hermano`, `hermana` y `tio`

Enunciado. Definir en Prolog, sobre el árbol genealógico de la familia Simpson, las relaciones `hermano(X, Y)`, `hermana(X, Y)` y `tio(X, Y)`, donde X es hermano, hermana o tío de Y, respectivamente. Las tres se construyen a partir de los hechos `hombre/1`, `mujer/1` y `progenitor/2`, que se cargan desde el archivo `familia.pl`. Las reglas se escribieron en el archivo `hermanos_tios.pl`.

Razonamiento en lenguaje natural. Dos personas son hermanos cuando son distintas y tienen al menos un progenitor en común. Si X es hombre, X es hermano de Y, y si X es mujer, X es hermana de Y. Por otro lado, X es tío de Y cuando X es hermano de alguno de los progenitores de Y. Por ejemplo, Herb y Homero tienen como progenitor a Abraham, por lo que son hermanos; como Homero es progenitor de Bart, Herb es tío de Bart.

Código en Prolog:

ejercicio1 > hermanos_tios.pl

```
1  :- consult('familia.pl').
2
3  hermano(X, Y) :- hombre(X), progenitor(Z, X), progenitor(Z, Y), X \= Y.
4  hermana(X, Y) :- mujer(X), progenitor(Z, X), progenitor(Z, Y), X \= Y.
5  tio(X, Y) :- progenitor(Z, Y), hermano(X, Z).
```

Figura 2: Reglas `hermano`, `hermana` y `tio` en el archivo `hermanos_tios.pl`.

Explicación paso a paso.

- `:- consult('familia.pl').` carga los hechos `hombre/1`, `mujer/1` y `progenitor/2` del árbol genealógico, para que las reglas puedan utilizarlos.
- `hombre(X)` comprueba que X sea hombre. En `hermana` se usa `mujer(X)`, que comprueba que X sea mujer.

- `progenitor(Z, X), progenitor(Z, Y)` busca un Z que sea progenitor de X y de Y al mismo tiempo, es decir, un progenitor en común.
- `X \= Y` exige que X y Y sean personas distintas. Sin esta condición, cada persona resultaría hermano o hermana de sí misma.
- En `tio`, `progenitor(Z, Y)` toma un progenitor Z de Y, y `hermano(X, Z)` comprueba que X sea hermano de Z.

Respuestas repetidas. Si dos hermanos comparten a sus dos progenitores, la misma pareja se obtiene una vez por cada progenitor en común. Por ejemplo, Bart y Lisa comparten a Homero y a Marge. Para mostrar cada resultado una sola vez se utilizó `setof/3`, que además ordena los resultados y elimina los repetidos.

Consultas y resultados. En la siguiente tabla se resumen las consultas realizadas.

Consulta	Resultado	Explicación
`hermano(X, Y)`	4 parejas	Bart es hermano de Lisa y de Maggie; Herb y Homero son hermanos porque ambos tienen a Abraham como progenitor.
`hermana(X, Y)`	10 parejas	Marge, Patty y Selma son hermanas entre sí (6 parejas); Lisa y Maggie son hermanas entre sí y de Bart (4 parejas).
`tio(X, Y)`	3 parejas	Herb es tío de Bart, Lisa y Maggie.
`tio(herb, bart)`	`true`	Herb es hermano de Homero, que es progenitor de Bart.

Comprobación en Prolog.

```
7 ?- setof(X-Y, hermano(X, Y), L), forall(member(P, L), (write(P), nl)).
bart-lisa
bart-maggie
herb-homero
homero-herb
L = [bart-lisa, bart-maggie, herb-homero, homero-herb].

8 ?- setof(X-Y, hermana(X, Y), L), forall(member(P, L), (write(P), nl)).
lisa-bart
lisa-maggie
maggie-bart
maggie-lisa
marge-patty
marge-selma
patty-marge
patty-selma
selma-marge
selma-patty
L = [lisa-bart, lisa-maggie, maggie-bart, maggie-lisa, marge-patty, marge-selma, patty-marge, patty-selma, … - …|…].
```

Figura 3: Resultado de las consultas `hermano(X, Y)` y `hermana(X, Y)` en Prolog.

En la figura 3, la consulta `hermano(X, Y)` devolvió 4 parejas: `bart-lisa`, `bart-maggie`, `herb-homero` y `homero-herb`. La consulta `hermana(X, Y)` devolvió 10 parejas, que corresponden a las hermanas del árbol: Marge, Patty y Selma entre sí, y Lisa y Maggie entre sí y con Bart.

```
9 ?- setof(X-Y, tio(X, Y), L), forall(member(P, L), (write(P), nl)).
herb-bart
herb-lisa
herb-maggie
L = [herb-bart, herb-lisa, herb-maggie].
```

Figura 4: Resultado de la consulta `tio(X, Y)` en Prolog.

En la figura 4, la consulta devolvió 3 parejas: `herb-bart`, `herb-lisa` y `herb-maggie`.

```
13 ?- tio(herb, bart).
true .

14 ?- hermano(bart, lisa).
true .

15 ?- hermana(lisa, bart).
true .
```

Figura 5: Consultas de verificación en Prolog.

En la figura 5, las tres consultas respondieron `true`. Esto indica que Prolog logró demostrarlas con los hechos del árbol: Herb es hermano de Homero y Homero es progenitor de Bart, y Bart y Lisa comparten a Homero como progenitor.

Observación. Herb es el único tío del árbol. Clancy solo tiene hijas (Marge, Patty y Selma), por lo que de ese lado no hay tíos hombres y Ling tiene tías pero no tíos.

1.5. Ejercicio 1: relaciones `tia`, `sobrino` y `sobrina`

Enunciado. Definir las relaciones `tia(X, Y)`, `sobrino(X, Y)` y `sobrina(X, Y)`, donde X es tía, sobrino o sobrina de Y, respectivamente. Las reglas se escribieron en el archivo `tias_sobrinos.pl`, que carga `hermanos_tios.pl` para utilizar las relaciones `hermana` y `tio` definidas anteriormente.

Razonamiento en lenguaje natural. X es tía de Y cuando X es hermana de alguno de los progenitores de Y. X es sobrino de Y cuando X es hombre y Y es su tío o su tía. La relación sobrina se define de la misma forma, pero X debe ser mujer. Por ejemplo, Patty es hermana de Marge y Marge es progenitora de Bart, por lo que Patty es tía de Bart y Bart es sobrino de Patty.

Código en Prolog:

```
:- consult('hermanos_tios.pl').

% tia(X, Y): X es tia de Y.
tia(X, Y) :-
    progenitor(Z, Y),
    hermana(X, Z).

% sobrino(X, Y): X es sobrino de Y.
sobrino(X, Y) :-
    hombre(X),
    tio(Y, X).
sobrino(X, Y) :-
    hombre(X),
    tia(Y, X).

% sobrina(X, Y): X es sobrina de Y.
sobrina(X, Y) :-
    mujer(X),
    tio(Y, X).
sobrina(X, Y) :-
    mujer(X),
    tia(Y, X).
```

Explicación paso a paso.

- `:- consult('hermanos_tios.pl').` carga las reglas `hermano`, `hermana` y `tio`, y con ellas los hechos de `familia.pl`.
- `tia` sigue la misma idea que `tio`: `progenitor(Z, Y)` toma un progenitor Z de Y, y `hermana(X, Z)` comprueba que X sea hermana de Z.
- `sobrino` tiene dos reglas: la primera se cumple cuando Y es tío de X y la segunda, cuando Y es tía de X. Prolog prueba las dos en orden. La relación `sobrina` funciona igual.
- `hombre(X)` en `sobrino` y `mujer(X)` en `sobrina` distinguen entre sobrino y sobrina.
- Estas relaciones son inversas a las de tío y tía: si Patty es tía de Bart, entonces Bart es sobrino de Patty.

Respuestas repetidas. Marge, Patty y Selma comparten a sus dos progenitores, por lo que `hermana` devuelve cada una de esas parejas dos veces y `tia` conserva esa repetición. Por eso, en la consulta `tia(X, ling)`, Marge y Patty aparecen dos veces cada una. Al igual que en la sección anterior, para contar los resultados sin repetir se utilizó `setof/3`.

Consultas y resultados. Se realizaron las siguientes consultas en SWI-Prolog:

```
?- tia(patty, bart).
true ;
true.

?- tia(X, ling).
X = marge ;
X = marge ;
X = patty ;
X = patty ;
false.

?- sobrino(bart, herb).
true ;
false.

?- sobrina(ling, marge).
true ;
true.

?- tio(homero, ling).
false.
```

Consulta	Resultado	Explicación
`tia(X, Y)`	8 parejas	Patty y Selma son tías de Bart, Lisa y Maggie; Marge y Patty son tías de Ling.
`sobrino(X, Y)`	3 parejas	Bart es sobrino de Herb, Patty y Selma.
`sobrina(X, Y)`	8 parejas	Lisa y Maggie son sobrinas de Herb, Patty y Selma; Ling es sobrina de Marge y Patty.

Observación. La consulta `tio(homero, ling)` respondió `false`. Homero es esposo de Marge, que es hermana de Selma, pero no es hermano de Selma. Como el árbol solo registra relaciones de progenitor y no matrimonios, las reglas consideran únicamente a los tíos y tías de sangre.

1.6. Ejercicio 2: hechos del grafo

Enunciado. Representar en Prolog el grafo dirigido con vértices $\{a, b, c, d\}$ y arcos $\{(a, b), (a, c), (a, d), (c, d)\}$.

Razonamiento en lenguaje natural. Cada arco del grafo es una flecha que va de un vértice a otro. Se representa con un hecho `arco(X, Y)`, que indica que existe un arco desde X hasta Y. Los vértices no necesitan hechos propios, porque aparecen dentro de los arcos.

Código en Prolog: Los hechos se escribieron al inicio del archivo `grafo.pl`.

```
% Ejercicio 2: grafo dirigido ({a,b,c,d}, {(a,b),(a,c),(a,d),(c,d)})

% arco(X, Y): existe una arista dirigida de X a Y.
arco(a, b).
arco(a, c).
arco(a, d).
arco(c, d).
```

Explicación paso a paso.

- Hay un hecho por cada arco del grafo, en el mismo orden del enunciado.
- El grafo es dirigido, por lo que `arco(a, b)` no implica que exista un arco de b hacia a.
- De los vértices b y d no sale ningún arco. Solo reciben arcos.
- Prolog revisa los hechos de arriba hacia abajo, por lo que este orden determina el orden de las soluciones y de las ramas del árbol de búsqueda.

Consultas y resultados. Se realizaron las siguientes consultas en SWI-Prolog:

```
?- arco(a, X).
X = b ;
X = c ;
X = d.

?- arco(X, d).
X = a ;
X = c.

?- arco(d, X).
false.
```

La primera consulta devuelve los tres vértices a los que se llega desde a, en el orden en que están escritos los hechos. La segunda devuelve los vértices que tienen un arco directo hacia d. La última respondió `false` porque de d no sale ningún arco.

1.7. Ejercicio 2: definición del predicado `camino`

Enunciado. Definir el predicado `camino/3` para encontrar caminos en el grafo dirigido con vértices $\{a, b, c, d\}$ y arcos $\{(a, b), (a, c), (a, d), (c, d)\}$.

Razonamiento en lenguaje natural. Existe un camino entre dos vértices si hay un arco directo entre ellos, o si se puede avanzar a un vértice intermedio y continuar desde allí hasta el destino. Estas posibilidades se representan con un caso base y un caso recursivo.

Código en Prolog: Las siguientes reglas se agregaron al archivo `grafo.pl`, después de los hechos `arco/2`.

```
% camino(X, Y, C): C es un camino desde X hasta Y.

% Caso base: hay un arco directo.
camino(X, Y, [X, Y]) :-
    arco(X, Y).

% Caso recursivo: pasar por un vertice intermedio Z.
camino(X, Y, [X | Resto]) :-
    arco(X, Z),
    camino(Z, Y, Resto).
```

Explicación paso a paso.

- En `camino(X, Y, C)`, X es el vértice inicial, Y es el destino y C es la lista de vértices que forman el camino.
- El caso base comprueba si existe `arco(X, Y)`. Si existe, el camino es la lista `[X, Y]`.
- El caso recursivo busca un arco desde X hasta un vértice intermedio Z. Luego llama nuevamente a `camino` para buscar cómo llegar desde Z hasta Y.
- En `[X | Resto]`, X es el primer elemento de la lista y `Resto` contiene el camino encontrado desde Z hasta el destino.

Ejemplo. Para construir el camino `[a,c,d]`, la regla recursiva utiliza el arco de a hacia c. Después busca un camino de c hacia d. Como existe un arco directo entre estos vértices, el caso base obtiene `[c,d]`. Finalmente, se coloca a al inicio de la lista y se forma `[a,c,d]`.

Observación. Esta definición encuentra caminos de al menos un arco y es suficiente para el grafo de la tarea, que no contiene ciclos. Las consultas y sus resultados se presentan en el siguiente apartado.

1.8. Ejercicio 2: ejecución de la consulta `camino(X, d, Y)`

Enunciado. Se tiene el grafo dirigido $(\{a, b, c, d\}, \{(a, b), (a, c), (a, d), (c, d)\})$, cuyos arcos y el predicado `camino` se encuentran en el archivo `grafo.pl`. Se debe ejecutar la consulta `camino(X, d, Y)`, donde X es un vértice de inicio y Y es el camino, y obtener todas sus soluciones.

Consulta en Prolog:

```
camino(X, d, Y).
```

Después de cada respuesta se pulsa `;` para que Prolog retroceda y busque otra solución, hasta que responde `false`. Para ver todas las soluciones juntas se utilizó:

```
forall(camino(X, d, Y), (write(X-Y), nl)).
```

Resultados. Prolog devolvió tres soluciones:

X	Y	Interpretación
a	[a, d]	Desde a se llega a d directamente, por el arco (a, d).
c	[c, d]	Desde c se llega a d directamente, por el arco (c, d).
a	[a, c, d]	Desde a se llega a c y desde c se llega a d.

Comprobación en Prolog.

```
PS C:\Users\sosad\OneDrive\Documentos\proyec3-logica\ejercicio2> swipl grafo.pl
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

1 ?- camino(X, d, Y).
X = a,
Y = [a, d]
Unknown action:  (h for help)
Action? ;
X = c,
Y = [c, d]
Unknown action:  (h for help)
Action? ;
X = a,
Y = [a, c, d]
Unknown action:  (h for help)
Action? ;
false.
```

Figura 6: Soluciones de `camino(X, d, Y)` obtenidas una por una.

En la figura 6, Prolog muestra las tres soluciones y, después de la tercera, responde `false`, lo que indica que no existen más caminos que terminen en `d`.

```
2 ?- forall(camino(X, d, Y), (write(X-Y), nl)).
a-[a,d]
c-[c,d]
a-[a,c,d]
true.
```

Figura 7: Todas las soluciones de `camino(X, d, Y)` en una sola consulta.

En la figura 7, las tres soluciones se muestran juntas, una por línea.

Observación. Los vértices `b` y `d` no aparecen como inicio de ningún camino hacia `d`, porque de ellos no sale ningún arco. Las tres soluciones obtenidas coinciden con las tres hojas de éxito del árbol de búsqueda de la consulta.

1.9. Ejercicio 2: árbol de búsqueda de `camino(X, d, Y)`

Enunciado. Graficar el árbol de búsqueda que recorre Prolog para resolver la consulta `camino(X, d, Y)`, utilizando un programa.

Programa utilizado. El árbol se generó con el programa `generar_arbol.py`, escrito en Python, en lugar de SldDraw, que el enunciado menciona como ejemplo. El programa lee las reglas y los hechos del archivo `grafo.pl` y recorre la búsqueda en el mismo orden que Prolog: prueba primero el caso base y después el caso recursivo, y revisa los hechos `arco/2` de arriba hacia abajo. El resultado es la imagen `arbol_busqueda.png`, que se muestra en la figura 8.

Cómo leer el árbol.

- La raíz es la consulta `camino(X,d,Y)`. Cada nodo muestra los objetivos que Prolog todavía debe demostrar.
- Cada rama corresponde a una regla o a un hecho que coincide con el objetivo. La etiqueta indica qué caso se usó (`base` o `rec.`) y qué valores tomaron las variables.
- Las hojas verdes son éxitos: ya no quedan objetivos y se obtiene una solución. Las hojas rojas son fallos: ningún hecho coincide con el objetivo.
- `Z1`, `R1`, `Z2`, `R2` y `Z3`, `R3` son nombres nuevos que reciben las variables cada vez que Prolog vuelve a usar la regla recursiva, para no confundirlas con las de la llamada anterior.

Explicación paso a paso. Para recorrer el árbol, Prolog utiliza dos mecanismos. El primero es la unificación, que consiste en hacer coincidir un objetivo con un hecho o con el inicio de una regla, asignando valores a las variables. El segundo es el retroceso (*backtracking*), que consiste en regresar a la última decisión en la que quedaba otra opción, ya sea porque una rama falló o porque se pidió otra solución con `;`.

1. Prolog toma el objetivo `camino(X, d, Y)` y lo unifica con el caso base, `camino(X, Y, [X, Y])`. Para que coincidan, Y debe ser la lista `[X, d]`, y el nuevo objetivo es `arco(X, d)`.
2. Prolog revisa los hechos `arco/2` en orden. `arco(a, b)` y `arco(a, c)` no coinciden, porque su segundo vértice no es d. `arco(a, d)` sí coincide con X = a, así que se obtiene la primera solución: X = a, Y = [a, d].
3. Al pulsar `;`, Prolog retrocede y sigue revisando los hechos desde donde se quedó. `arco(c, d)` coincide con X = c y se obtiene la segunda solución: X = c, Y = [c, d].
4. Ya no quedan hechos que coincidan con `arco(X, d)`, así que Prolog retrocede hasta la consulta inicial y prueba el caso recursivo. Ahora Y = [X | R1] y los objetivos pendientes son `arco(X, Z1)` y `camino(Z1, d, R1)`.

5. El primer hecho que coincide con `arco(X, Z1)` es `arco(a, b)`, con X = a y Z1 = b. Prolog debe demostrar `camino(b, d, R1)`. El caso base necesita `arco(b, d)` y el caso recursivo necesita un arco que salga de b. Ninguno existe, por lo que las dos ramas fallan y Prolog retrocede.

6. El siguiente hecho es `arco(a, c)`, con Z1 = c. Para `camino(c, d, R1)`, el caso base encuentra `arco(c, d)`, así que R1 = [c, d] y, por lo tanto, Y = [a, c, d]. Esta es la tercera solución.

7. Al pulsar `;` de nuevo, Prolog prueba el caso recursivo de `camino(c, d, R1)`. Usa `arco(c, d)` con Z2 = d y debe demostrar `camino(d, d, R2)`. Como de d no sale ningún arco, ni el caso base ni el recursivo encuentran hechos y la rama falla.

8. Prolog retrocede a `arco(X, Z1)` y usa `arco(a, d)`, con Z1 = d. Otra vez debe demostrar `camino(d, d, R1)`, que falla por la misma razón.

9. Por último, usa `arco(c, d)`, con X = c y Z1 = d. Se repite el objetivo `camino(d, d, R1)`, que también falla.

10. No quedan opciones por probar en ninguna rama, así que Prolog responde `false`.

En total, el árbol tiene tres hojas de éxito, que coinciden con las tres soluciones obtenidas en la sección 1.8, y ocho hojas de fallo.

Observación. El orden de los hechos en `grafo.pl` determina el orden de las soluciones. Por ejemplo, si `arco(c, d)` estuviera escrito antes que `arco(a, d)`, la primera solución sería X = c, Y = [c, d].

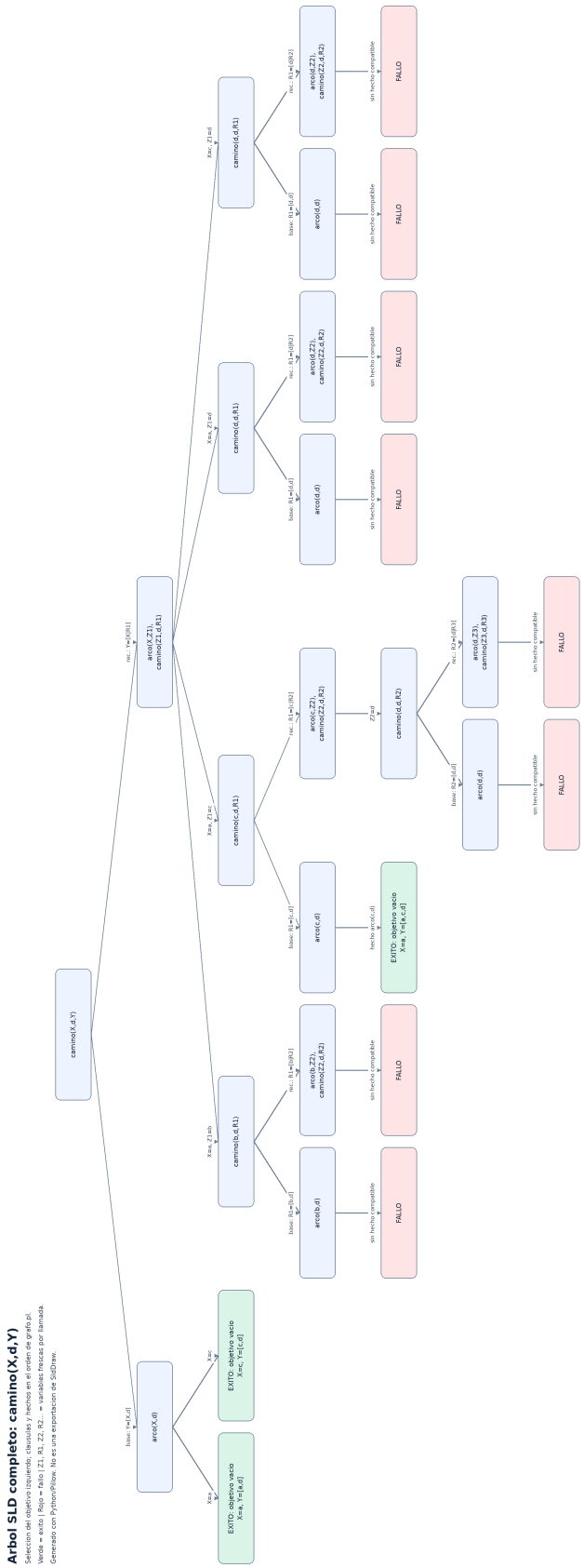


Figura 8: Árbol de búsqueda de la consulta `camino(X, d, Y)`.

2. Archivos entregados

Archivo	**Contenido**
`ejercicio1/familia.pl`	Hechos `hombre`, `mujer` y `progenitor`, y reglas `padre`, `madre`, `hijo`, `hija`, `abuelo` y `abuela`.
`ejercicio1/hermanos_tios.pl`	Reglas `hermano`, `hermana` y `tio`.
`ejercicio1/tias_sobrinos.pl`	Reglas `tia`, `sobrino` y `sobrina`.
`ejercicio1/arbol.png`	Representación gráfica del árbol genealógico.
`ejercicio1/arbol.dot`	Archivo con el que se genera el dibujo del árbol genealógico.
`ejercicio2/grafo.pl`	Hechos `arco` y predicado `camino`.
`ejercicio2/consulta_camino.pl`	Consulta `camino(X, d, Y)`.
`ejercicio2/generar_arbol.py`	Programa que genera el árbol de búsqueda.
`ejercicio2/arbol_busqueda.png`	Árbol de búsqueda de la consulta `camino(X, d, Y)`.
`ejercicio2/arbol_busqueda.dot`	Descripción del árbol de búsqueda en formato DOT.

Para utilizar todas las relaciones del Ejercicio 1 basta con cargar `tias_sobrinos.pl` en SWI-Prolog, ya que este archivo carga `hermanos_tios.pl` y, a su vez, `familia.pl`.